

Artificial Intelligence, Blockchain e Criptovalute nello Sviluppo Software

Manuale Narrativo d'Esame

Prof. Giancarlo Succi · Università di Bologna · LM Informatica · Primavera 2026

Testo elaborato per lo studio orale — spiegazioni narrative e ragionate

■ COME USARE QUESTO MANUALE

Questo documento è pensato per essere **letto come un libro**, non consultato come una scheda. Ogni capitolo espone i concetti in modo progressivo, con spiegazioni estese, esempi e ragionamenti. L'obiettivo è che tu possa presentarti all'orale con una comprensione profonda, non con definizioni imparate a memoria.

Il Prof. Succi apprezza studenti che **ragionano e collegano i concetti**. Prepara ogni risposta con almeno un collegamento a un altro tema del corso. Le tabelle sintetiche del manuale precedente rimangono utili per il ripasso rapido dell'ultima sera, ma la comprensione si costruisce leggendo queste pagine.

■ L'Ingegneria del Software — Fondamenta Concettuali

CAPITOLO 1

La Natura del Software e la sua Economia

Cosa rende il software diverso da qualsiasi altro prodotto?

Per rispondere a questa domanda bisogna partire da una constatazione apparentemente semplice, ma dalle implicazioni profonde: **il software è il risultato di un atto creativo della mente umana**. Non si tratta di assemblare componenti fisici, di seguire una ricetta predefinita, o di replicare un processo manifatturiero. È più simile a scrivere un romanzo o comporre una sinfonia che a costruire un'automobile. Questa analogia non è retorica: ha conseguenze dirette su come il software viene protetto legalmente, distribuito economicamente e sviluppato organizzativamente.

Dal punto di vista legale, il software è protetto dal **diritto d'autore** (copyright), non dal brevetto. La differenza è sostanziale: un brevetto dura circa 20 anni dalla registrazione e richiede una procedura formale; il copyright protegge l'opera automaticamente, dal momento della sua creazione, per **50 anni dopo la morte dell'autore**. I diritti si dividono in due categorie: i **diritti morali**, che riguardano il riconoscimento della paternità dell'opera e sono *non alienabili* (un programmatore non può rinunciare ad essere l'autore di ciò che ha scritto, neanche se viene pagato per farlo), e i **diritti economici**, che riguardano lo sfruttamento commerciale e sono invece *alienabili*, cioè trasferibili a un'azienda o a un datore di lavoro. Questo è il motivo per cui il software non viene "venduto" all'utente finale: viene concesso in **licenza d'uso**. Quando acquisti una copia di Windows, non stai comprando il programma — stai comprando il diritto di usarlo secondo i termini stabiliti dal produttore.

L'economia del software: la funzione di costo a "L"

Il carattere immateriale del software genera una struttura economica senza paragoni nell'industria tradizionale. Per sviluppare un programma complesso servono mesi o anni di lavoro altamente specializzato: il **costo fisso** di sviluppo è elevatissimo. Ma, una volta completato, il **costo marginale** — cioè il costo per produrre una copia aggiuntiva — è praticamente zero. Duplicare un file non costa nulla. Questo produce quella che viene chiamata una funzione di costo a forma di "L": un muro verticale iniziale (il costo di sviluppo), seguito da una retta orizzontale quasi piatta (il costo delle copie successive).

Questa struttura ha conseguenze strategiche notevoli. Rende possibile una **segmentazione aggressiva del mercato**: uno stesso prodotto può essere venduto a prezzi molto diversi a segmenti diversi di utenti — versione professionale, educativa, gratuita con funzioni limitate — senza che il costo di produzione cambi significativamente. Rende anche conveniente il **bundling**, cioè il pacchettizzare più prodotti insieme per promuovere quelli nuovi sfruttando la notorietà di quelli esistenti. Infine, genera **economie di rete**: il valore di un prodotto software cresce con il numero degli utenti (un sistema operativo che nessuno usa non ha valore,

anche se tecnicamente perfetto).

■■ ATTENZIONE — ERRORE COMUNE ALL'ESAME

Un errore frequente è confondere il fatto che nel software esistano economie di scala (replicare è gratis) con il fatto che il modello industriale Fordista funzioni nel software. La produzione in lotti abbate il costo medio nel manifatturiero perché riduce il costo di lavorazione per unità. Nel software il costo di lavorazione della replica è già zero, quindi il Fordismo non aggiunge valore. Anzi, la sua rigidità organizzativa diventa un problema di fronte all'incertezza intrinseca del software.

CAPITOLO 2

La Software Crisis e le Quattro Difficoltà di Brooks

Come nacque la consapevolezza di una crisi

Negli anni '60 e '70 del Novecento, man mano che i calcolatori diventavano più potenti, ci si aspettava che anche i programmi per gestirli diventassero proporzionalmente più facili da costruire. Accadde esattamente il contrario. Il matematico e informatico olandese Edsger Dijkstra, nel 1972, coniò l'espressione "**software crisis**" per descrivere un fenomeno preoccupante: la potenza hardware cresceva esponenzialmente, ma la capacità umana di produrre software affidabile, nei tempi e nei costi previsti, non riusciva a stare al passo. I progetti software venivano consegnati in ritardo, con costi sforati, pieni di bug e spesso non rispondenti alle reali esigenze degli utenti. Questa situazione non era una fase transitoria destinata a risolversi: era strutturale.

La risposta intellettuale più influente arrivò da **Fred Brooks**, che nel suo celebre saggio "No Silver Bullet" (1986) identificò quattro difficoltà fondamentali e intrinseche del software — ovvero difficoltà che non possono essere eliminate da nessuna innovazione tecnologica, perché derivano dalla natura stessa della cosa.

Le quattro difficoltà strutturali

La prima è la **Complessità**. Un programma ha un numero di stati possibili — combinazioni di valori di variabili, condizioni di esecuzione, flussi di controllo — che supera di gran lunga quello di qualsiasi sistema fisico. Una macchina ha milioni di parti, ma in un certo senso ciascuna fa solo una cosa. Un sistema software di medie dimensioni può avere miliardi di cammini di esecuzione diversi, ognuno potenzialmente portatore di comportamenti inattesi. Questa complessità non è eliminabile: è intrinseca alla natura computazionale del software.

La seconda difficoltà è la **Conformità**. Il software non esiste nel vuoto: deve interfacciarsi con sistemi preesistenti, leggi, regolamenti, organizzazioni e aspettative di persone che spesso non cambiano, o cambiano in modo imprevedibile. Un sistema bancario deve rispettare normative che cambiano ogni anno, interfacciarsi con protocolli legacy degli anni '70 e adattarsi alle abitudini degli utenti. Il software deve *conformarsi* a un mondo che non è stato progettato per lui, e spesso questo mondo è irrazionale, contraddittorio o semplicemente già cristallizzato in forme che non si vogliono toccare.

La terza è la **Modificabilità**. Paradossalmente, il software di successo è quello che viene cambiato di più. Se un programma piace agli utenti, questi ne chiedono l'estensione, l'adattamento, l'integrazione con nuove

funzionalità. A differenza di un edificio — dove modificare i muri portanti è costoso e visibile — il software sembra infinitamente malleabile, e questa percezione porta a richieste di cambiamento continue e spesso mal pianificate. Il software "vivo" è costantemente sotto pressione evolutiva.

La quarta difficoltà è l'**Invisibilità**. Il software è immateriale: non si vede, non si tocca, non ha una forma fisica. Questa caratteristica rende estremamente difficile comunicare la struttura di un sistema a chi non lo conosce, visualizzarne l'architettura, o percepire intuitivamente dove si trovano i problemi. Un ingegnere civile può guardare un edificio e capire molto del suo stato. Un project manager che guarda un sistema software vede al massimo del codice che scorre su uno schermo, senza poter intuire la complessità nascosta sotto.

Il caso Ariane 5: una lezione da ricordare

Per rendere concrete queste difficoltà, il docente Succi cita frequentemente il **disastro di Ariane 5** del 1996. Il vettore spaziale europeo, al suo primo volo, si autodistrusse 37 secondi dopo il lancio. La causa? Un modulo software — il sistema di riferimento inerziale — era stato preso direttamente dal vettore precedente, Ariane 4, dove funzionava perfettamente. Il problema era che Ariane 5 aveva una traiettoria di lancio diversa, con velocità orizzontali molto più elevate. Questo causò un **overflow aritmetico**: un numero in virgola mobile a 64 bit venne inserito in una variabile a 16 bit, che non era in grado di contenerlo. Il sistema andò in errore, mandò un codice diagnostico erraneo all'unità di controllo principale, e il razzo si autodistrusse. I danni furono stimati intorno agli 8 miliardi di dollari.

La lezione di Ariane 5 non è semplicemente "testa meglio il tuo codice". È molto più profonda: il software non è un componente fisico intercambiabile. Riutilizzare un modulo software in un contesto diverso da quello per cui è stato progettato è rischioso esattamente perché il software deve **conformarsi** al contesto (seconda difficoltà di Brooks), e la sua **invisibilità** (quarta difficoltà) rende difficile percepire che il contesto è cambiato in modo critico.

■ COME RISPONDERE ALL'ESAME

Quando il docente chiede di questo argomento, non si aspetta solo l'elenco delle quattro difficoltà. Si aspetta che tu le colleghi a casi reali e che mostri di aver capito perché sono "strutturali" — cioè perché non basta un nuovo linguaggio di programmazione o un nuovo tool per eliminarle. La risposta ideale è: spiegare la difficoltà, dare un esempio concreto, e poi dire perché non è risolvibile con un "silver bullet" tecnologico.

CAPITOLO 3

La Teoria dei Sistemi Dinamici e i Regimi di Mensch

Perché il software non si comporta come ci aspettiamo

Uno dei contributi teorici più originali del corso riguarda la classificazione dei sistemi dinamici secondo i regimi descritti da Mensch. L'idea di partenza è semplice ma potente: il software non è un sistema meccanico deterministico. Non si comporta sempre come previsto, e confondere il "tipo di sistema" che si sta gestendo è, secondo Succi, **la causa principale dei fallimenti nel software**. Capire in che regime si trova il proprio sistema permette di scegliere l'approccio di gestione corretto.

Il primo regime è quello **Pseudo-Deterministico (PDS)**. Un sistema PDS sembra stabile e prevedibile perché, nella maggior parte delle condizioni operative normali, si comporta regolarmente. Ma questa stabilità non è una proprietà intrinseca del sistema: è ottenuta tramite ingegneria, progettazione istituzionale o meccanismi di controllo. Nasconde fragilità latenti che emergono solo quando si verificano perturbazioni fuori dall'ordinario. Il caso più evocativo per spiegare questo regime è la "**illusione del tacchino**": un tacchino allevato viene nutrito ogni giorno dal contadino. Con il passare del tempo, costruisce un modello mentale del mondo in cui il contadino è una fonte affidabile di cibo. Questo modello PDS funziona perfettamente per 364 giorni. Il giorno del Ringraziamento, il tacchino scopre che la sua previsione era catastroficamente sbagliata. Nel software, molti sistemi in produzione sono PDS: sembrano stabili finché non incontrano un caso limite, un picco di traffico anomalo, o un cambiamento dell'ambiente esterno.

Il secondo regime è quello **Caotico Deterministico (DCS)**. Questo regime è governato da equazioni precise e completamente deterministiche, ma presenta una **sensibilità estrema alle condizioni iniziali**. Due esecuzioni dello stesso sistema con parametri iniziali infinitesimalmente diversi producono risultati radicalmente diversi nel lungo periodo. Questo è il famoso "effetto farfalla" di Lorenz: una farfalla che batte le ali in Brasile può, in teoria, influenzare il percorso di un uragano in Texas. Nel software, i bug di tipo DCS sono i più difficili da riprodurre: si manifestano solo in certe condizioni, con certi dati, in certi momenti — e non si ripresentano quasi mai identici. I race condition nei sistemi concorrenti sono un esempio tipico.

Il terzo regime è quello **Stocastico (StS)**. Qui c'è incertezza strutturale nel comportamento del sistema, ma questa incertezza mantiene una struttura probabilistica stabile. Non si può prevedere il singolo evento, ma si può stimare con ragionevole accuratezza la distribuzione degli eventi. Questo è il regime per il quale sono nati i metodi statistici di gestione del rischio. Molti processi software, specialmente quelli Agile ben gestiti, operano in regime StS: non si può sapere esattamente quanto durerà una user story, ma si può stimarne la distribuzione sulla base della velocità storica del team.

Il quarto regime è la **Randomicità Stocastica (SR)**, che rappresenta il caos puro. Non esiste struttura probabilistica stabile: la previsione è impossibile, e la sola strategia possibile è il contenimento dei danni. Un cyberattacco sofisticato in un sistema compromesso può portare il comportamento del software in regime SR.

■ NOTA DI APPROFONDIMENTO

Il collegamento con i regimi di Mensch è il filo rosso di tutto il corso. Il Taylorismo e il modello Waterfall assumono implicitamente che il software sia in regime PDS: il processo è pianificabile in dettaglio perché si comporterà come previsto. Questa assunzione è quasi sempre sbagliata. L'Agile è la risposta a un software che è DCS o StS: invece di pianificare rigidamente, si usano cicli brevi di feedback per navigare l'incertezza. La blockchain, invece, è un sistema ingegneristico che cerca di costruire un regime PDS artificiale in un ambiente distribuito e potenzialmente ostile, usando il protocollo di consenso come meccanismo di retroazione negativa.

CAPITOLO 4

Problemi Tame e Wicked

Non tutti i problemi sono uguali

Nel 1973, i ricercatori Horst Rittel e Melvin Webber pubblicarono un articolo destinato a diventare un classico nelle scienze della progettazione: "Dilemmas in a General Theory of Planning". In quel lavoro introducevano la distinzione tra problemi **Tame** (mansueti, docili) e problemi **Wicked** (perversi, ostici). Questa distinzione non riguarda la difficoltà tecnica di un problema — un problema può essere tecnicamente facile eppure Wicked — ma la sua struttura epistemica: il modo in cui è possibile conoscerlo, formularlo e risolverlo.

Un problema **Tame** ha una formulazione stabile e definita. Esiste una soluzione che è chiaramente "giusta" o "sbagliata". Il problema appartiene a una classe di problemi simili, e il risolutore può affidarsi all'esperienza accumulata su problemi analoghi. Un'equazione differenziale, un puzzle, la costruzione di un ponte secondo specifiche precise — questi sono problemi Tame. L'approccio Waterfall e il Taylorismo presuppongono che il software sia un problema Tame: prima si raccolgono tutti i requisiti (formulazione stabile), poi si progetta (soluzione), poi si implementa (costruzione), poi si testa (verifica della correttezza).

Un problema **Wicked**, al contrario, non ha una formulazione definitiva. La comprensione del problema cambia con ogni tentativo di soluzione: ogni passo verso la risposta rivela nuovi aspetti del problema stesso che non erano visibili all'inizio. Le soluzioni non sono "vere" o "false", ma "**accettabili**" o "**non accettabili**" agli occhi degli stakeholder — e gli stakeholder non sono d'accordo su cosa significhi "accettabile". Ogni problema Wicked è essenzialmente unico: non appartiene a classi note in modo utile, perché il contesto specifico è sempre diverso. E ogni tentativo di soluzione ha conseguenze irreversibili, perché agisce su un sistema reale che non può essere riportato allo stato iniziale.

Il software è quasi invariabilmente un problema Wicked. Quando un cliente commissiona un'applicazione gestionale, non sa esattamente cosa vuole finché non vede i primi prototipi funzionanti. Ogni iterazione di sviluppo modifica la sua comprensione del proprio fabbisogno. I requisiti che sembravano stabili a luglio sono diventati obsoleti a settembre, perché nel frattempo l'azienda ha cambiato strategia, o un competitor ha lanciato un prodotto, o una normativa è cambiata. Non esiste una soluzione "giusta" in assoluto: esiste una soluzione che oggi soddisfa i principali stakeholder, con la consapevolezza che domani potrebbe non farlo più.

■ COME RISPONDERE ALL'ESAME

Una risposta eccellente a questa domanda collegherebbe la distinzione Tame/Wicked alla Matrice di Harris (Cap. 5) e ai regimi di Mensch (Cap. 3): i problemi Tame si gestiscono con controllo di tipo Behavior in regime PDS; i problemi Wicked richiedono controllo di tipo Clan/Group in regime StS. Questo collegamento mostra di aver capito il filo rosso del corso.

CAPITOLO 5

Coordinamento, Taylorismo e la Matrice di Harris

Come le persone producono software insieme

Malone e Crowston, nel loro studio sui meccanismi di coordinamento nelle organizzazioni, identificano tre modalità fondamentali con cui i gruppi umani possono strutturare il proprio lavoro collettivo. Comprendere queste modalità è essenziale per capire perché certi approcci di sviluppo software funzionano meglio di altri in certi contesti.

Il **coordinamento sequenziale** è il più semplice da comprendere: il risultato del lavoro di una persona o di un gruppo viene passato come input al gruppo successivo. È la logica della catena di montaggio applicata al software: analisti → progettisti → sviluppatori → tester → deployment. La chiarezza dei confini e la semplicità gestionale sono i vantaggi principali. Il limite è la rigidità: se un problema emerge tardi nella catena, il costo di tornare indietro è enorme.

Il **coordinamento per risorse condivise** prevede che tutti i membri del team attingano da una coda comune di task, secondo priorità definite. Questo è il modello di un backlog Agile gestito tramite Kanban: chiunque è libero di prendere il prossimo elemento prioritario quando ha finito quello corrente. La flessibilità è maggiore, ma richiede che le priorità siano esplicitamente definite e costantemente aggiornate.

Il **coordinamento per risultato comune** è il più impegnativo: tutto il team lavora contemporaneamente sullo stesso output, in sincronia. Il Pair Programming di XP è l'esempio per eccellenza. È il più difficile perché richiede unanimità — tutti devono essere allineati in tempo reale — ma è anche il più adatto ai problemi Wicked, perché permette di incorporare immediatamente il feedback di più prospettive nel processo creativo.

Taylor, Ford e i limiti del management classico nel software

Il **Taylorismo**, o Scientific Management, è il paradigma organizzativo che Frederick Taylor sviluppò nei primi del Novecento per massimizzare l'efficienza della produzione industriale. Il principio fondamentale è la separazione tra chi pensa (i manager e gli ingegneri che progettano il processo) e chi esegue (gli operai che seguono istruzioni precise). Ogni compito viene scomposto in movimenti elementari, il tempo di esecuzione viene misurato, e il processo viene ottimizzato. Questo funziona magnificamente quando il lavoro è ripetitivo, standardizzabile e quando l'operaio non deve capire il perché di ciò che fa, solo il come.

Il problema, applicato al software, è evidente: lo sviluppatore non è un operaio, e il suo lavoro non è ripetitivo. Scrivere codice richiede comprensione profonda del dominio, creatività, capacità di risolvere problemi nuovi e non standardizzabili. Un programmatore che non capisce perché sta scrivendo una certa funzione non può prendere le decisioni di micro-design che si presentano a ogni riga di codice. Il Taylorismo applicato al software produce le **specifiche rigide** del modello Waterfall — e sappiamo come spesso finisce.

La Matrice di Harris e la scelta del meccanismo di controllo

La Matrice di Harris è uno strumento concettuale che il docente cita esplicitamente come altamente rilevante per l'esame. Si tratta di una classificazione dei meccanismi di controllo di un processo, basata su quanto il processo è conoscibile e prevedibile.

Il controllo di tipo **Outcome** (per risultati) misura il risultato finale del processo. Funziona bene quando si conosce la relazione causa-effetto tra le azioni del team e il risultato desiderato. Un responsabile vendite che misura le vendite mensili sta usando un controllo Outcome: non gli interessa come il venditore ottiene i risultati, solo che li ottenga.

Il controllo di tipo **Behavior** (per comportamenti) misura i passaggi intermedi del processo. Funziona bene quando il processo è ben noto e standardizzabile — essenzialmente, quando siamo in un contesto Taylorista. Si misura quante ore di codice vengono scritte, quanti commit vengono fatti al giorno, quante review vengono completate. Il problema è che nel software creativo, misurare il comportamento è spesso controproducente: genera il fenomeno di Goodhart ("quando una misura diventa un obiettivo, cessa di essere una buona misura") e incentiva a ottimizzare la metrica invece del risultato reale.

Il controllo di tipo **Clan/Group** rinuncia a misurare direttamente né il risultato né il comportamento, e si affida invece alla fiducia reciproca, ai valori condivisi e all'auto-organizzazione del team. È il tipo di controllo implicito in tutti gli approcci Agile: il daily stand-up non misura nulla, ma crea allineamento; la retrospettiva non produce metriche, ma genera miglioramento continuo. Questo tipo di controllo è l'unico efficace quando il processo è talmente Wicked che non è possibile né definire un risultato preciso in anticipo né standardizzare i comportamenti che portano ad esso.

■ NOTA DI APPROFONDIMENTO

La Matrice di Harris è il ponte teorico che collega la distinzione Tame/Wicked con i metodi di sviluppo. Problemi Tame in regime PDS → controllo Behavior (Waterfall, Taylorismo). Problemi Wicked in regime StS → controllo Clan/Group (Agile). Questa mappa concettuale è fondamentale per rispondere alle domande del docente in modo integrato.

■ Modelli Cognitivi nella Produzione Software

CAPITOLO 6

Gregory Bateson e la Teoria della Percezione

Perché studiare la percezione per capire l'AI?

Uno dei contributi più originali e meno scontati del corso è la scelta di partire non dall'informatica, ma dalla filosofia della mente e dalla cibernetica per comprendere l'Intelligenza Artificiale. Gregory Bateson — biologo, antropologo e teorico dei sistemi — è la figura centrale di questa sezione. Il ragionamento del docente è il seguente: per capire cosa può fare un sistema di AI e quali sono i suoi limiti, dobbiamo prima capire come funziona l'intelligenza umana che quella AI cerca di modellare o emulare. E per capire come funziona l'intelligenza umana, dobbiamo partire da come l'uomo percepisce la realtà.

La definizione batesoniana di **informazione** è lapidaria e potentissima: *"una differenza che produce una differenza"*. Non tutti i dati sono informazione. Un segnale diventa informazione solo quando produce un cambiamento nel sistema che lo riceve. Se ti dico che ieri ha piovuto a Bogotá e tu non hai nessun interesse né connessione con quella città, quel dato non produce alcun cambiamento nel tuo sistema cognitivo: non è informazione per te. Se invece stai per partire per un viaggio a Bogotá, la stessa notizia è informazione altamente rilevante. Questa distinzione ha implicazioni profonde per il design dei sistemi software: non basta "avere i dati"; bisogna capire quali differenze quei dati producono nel sistema che li riceve.

Il secondo concetto fondamentale di Bateson è l'**abduzione**, che egli distingue dall'induzione e dalla deduzione come terza forma di ragionamento. L'induzione parte da casi particolari per arrivare a principi generali; la deduzione parte da principi generali per derivare casi particolari. L'abduzione, invece, è un ragionamento basato sulla ricerca e comparazione di **pattern**: di fronte a un insieme di osservazioni, si cerca il pattern — il modello, la struttura — che meglio le spiega. È il ragionamento del detective che, di fronte alle tracce di un delitto, cerca la narrativa più plausibile. È anche, fondamentalmente, il modo in cui funzionano i moderni Large Language Model: generano risposte cercando il pattern statistico più probabile dato il contesto. Questo non è "ragionamento" nel senso umano, ma è abduzione computazionale.

Il **double bind** di Bateson descrive una situazione comunicativa in cui due messaggi contraddittori a livelli logici diversi creano un paradosso irrisolvibile. L'esempio classico è "Sii spontaneo!": se sei spontaneo perché te lo hanno ordinato, non sei spontaneo. Nel contesto dello sviluppo software, il double bind si manifesta ogni volta che si chiede di "sviluppare velocemente e senza bug", oppure "essere creativi seguendo il processo".

La schismogenesi e la comunicazione nei team

Bateson coniò anche il termine **schismogenesi** per descrivere i processi di divergenza escalante tra individui o organizzazioni. Esistono due varietà. La **schismogenesi complementare** si verifica quando i comportamenti delle due parti si rinforzano in modo asimmetrico: il comportamento dominante di una parte

genera comportamento sottomesso nell'altra, che a sua volta rinforza il dominante, in un ciclo crescente. È la dinamica del gatto che insegue il topo: più il topo fugge, più il gatto insegue. Nei team di sviluppo, può manifestarsi come la relazione disfunzionale tra un senior molto autoritario e junior sempre più passivi. La **schismogenesi simmetrica**, al contrario, genera escalation simmetrica: il comportamento aggressivo di una parte genera risposta aggressiva nell'altra, in una spirale che tende all'esplosione. Il classico esempio è la corsa agli armamenti. Nei team, possono essere le discussioni tecniche che degenerano in conflitti personali.

I cinque assiomi della comunicazione di Watzlawick

Paul Watzlawick, della Scuola di Palo Alto, formulò cinque assiomi della comunicazione che hanno una rilevanza diretta per la gestione dei team di sviluppo. Il primo assioma afferma che **non è possibile non comunicare**: ogni comportamento, incluso il silenzio, è un messaggio. Uno sviluppatore che non risponde a un messaggio in chat sta comunque comunicando qualcosa — indifferenza, impegno su altro, disapprovazione — anche se non lo fa volontariamente. Il secondo assioma distingue tra il **contenuto** di un messaggio (cosa viene detto) e la **relazione** (come viene detto, chi lo dice a chi). La stessa frase — "questo bug va sistemato" — può essere un comando, una preghiera o una constatazione, a seconda del contesto relazionale. Il terzo assioma riguarda la **punteggiatura delle sequenze**: nelle interazioni circolari, ogni partecipante tende a vedere la propria risposta come reazione al comportamento altrui, e il comportamento altrui come causa del proprio. Il quarto distingue tra comunicazione **digitale** (le parole, il contenuto verbale) e **analogica** (il tono, la postura, le espressioni). Il quinto riconduce alla distinzione tra relazioni **simmetriche** (tra pari) e **complementari** (gerarchiche), con i rischi di schismogenesi che ognuna porta con sé.

CAPITOLO 7

Come Usiamo la Conoscenza: dai Modelli Classici al Modello Impulsivo/Riflessivo

L'evoluzione storica dei modelli cognitivi

La psicologia cognitiva e sociale ha sviluppato nel corso del Novecento una serie di modelli per descrivere come gli esseri umani elaborano le informazioni e prendono decisioni. Questi modelli si sono evoluti in modo interessante: ogni generazione di ricercatori ha scoperto che il modello precedente era troppo ottimistico sulle capacità razionali dell'essere umano.

Il modello più antico, chiamato del **Naive Scientist** (lo "scienziato ingenuo"), dominò gli anni '50 e '60. Secondo questo modello, l'essere umano si comporta come uno scienziato: raccoglie dati dall'ambiente in modo sistematico, formula ipotesi, le testa e aggiorna le proprie credenze di conseguenza. È un modello di razionalità ottimale, in cui l'individuo cerca sempre la migliore soluzione possibile sulla base delle informazioni disponibili. Applicato al software, questo sarebbe lo sviluppatore che legge tutta la documentazione, analizza tutti i requisiti prima di scrivere una riga di codice, e progetta l'architettura più elegante prima di implementarla.

Negli anni '80, la ricerca empirica mostrò che questo modello non corrispondeva al comportamento reale. Le persone non ottimizzano: *satisficono* (Simon, 1956). Nacque così il modello del **Cognitive Miser** — l'avaro cognitivo. L'idea è che le risorse cognitive (attenzione, memoria di lavoro, sforzo mentale) sono scarse e preziose. Per non esaurirle, il cervello usa continuamente **euristiche** — scorciatoie cognitive — che di solito portano a risposte accettabili con il minimo sforzo. Il programmatore che copia la soluzione da Stack Overflow senza capirla fino in fondo sta agendo da cognitive miser: non è pigrizia, è il normale

funzionamento cognitivo sotto pressione temporale.

Negli anni '90 si aggiunse un ulteriore livello di complessità con il modello del **Motivated Tactician**. Le ricerche mostrarono che gli esseri umani non sono né sempre ottimizzatori né sempre avari cognitivi: scelgono la strategia cognitiva in base alla motivazione, al contesto e all'importanza percepita del compito. Quando il posta è alta e la motivazione è forte, si investe nell'analisi razionale. Quando il contesto è routinario o le risorse sono limitate, si ricorre alle euristiche. Il programmatore esperto sa quando è il momento di applicare TDD rigoroso e quando è accettabile scrivere un piccolo script "quick and dirty".

Il Modello Impulsivo/Riflessivo: il cuore cognitivo del corso

Il modello più recente e più importante per questo corso è quello sviluppato da **Strack e Deutsch nel 2004**, pubblicato sotto il titolo "Reflective and impulsive determinants of social behavior". Questo modello — che riecheggia la distinzione Sistema 1 / Sistema 2 di Kahneman — descrive il comportamento umano come il risultato dell'interazione di **due sistemi cognitivi che operano in parallelo**.

Il **Sistema Impulsivo** è implicito, automatico e velocissimo. Opera in millisecondi, senza sforzo consapevole, sulla base di una memoria associativa costruita attraverso l'esperienza. Quando un programmatore esperto legge un blocco di codice e sente istintivamente che "qualcosa non va", sta usando il sistema impulsivo. Quando riconosce a colpo d'occhio un design pattern, o quando una riga di codice gli sembra immediatamente inefficiente senza che riesca a spiegare subito perché — tutto questo è il sistema impulsivo al lavoro.

Il **Sistema Riflessivo** è esplicito, deliberato e lento. Richiede sforzo cognitivo consapevole, attenzione focalizzata e tempo. È il sistema che si attiva quando analizziamo formalmente la complessità computazionale di un algoritmo, quando costruiamo mentalmente un proof of correctness, quando debugghiamo un race condition in un sistema concorrente tenendo traccia di tutte le possibili interleaving. Non si attiva automaticamente: deve essere "chiamato" con uno sforzo volontario.

I due sistemi operano in parallelo e a volte producono risultati coerenti, ma a volte si scontrano. L'esempio del dolce vs dieta, usato nelle slide originali del docente, illustra perfettamente il conflitto: il sistema riflessivo ha elaborato la valutazione "perdere peso è una cosa buona", il sistema impulsivo ha attivato l'orientamento motivazionale verso il dolce sul tavolo. Il comportamento finale dipende da quale sistema prevale in quel momento — e spesso, sotto pressione temporale o cognitiva, è il sistema impulsivo a vincere.

L'applicazione più importante di questo modello nel contesto del corso riguarda le **software inspection** e le **code review**. Le ricerche empiriche mostrano che i revisori di codice, anche quelli esperti, tendono a essere dominati dal sistema impulsivo quando il tempo a disposizione è limitato. Guardano il codice, "sentono" che va bene o che c'è un problema, e formulano un giudizio rapido. Il sistema riflessivo non viene attivato spontaneamente. Per migliorare la qualità delle review è necessario creare protocolli che *forzino* l'attivazione del sistema riflessivo: checklist esplicite da seguire passo dopo passo, separazione temporale tra la lettura e la valutazione, confronto tra più revisori. Senza questi protocolli, le review diventano fondamentalmente un esercizio del sistema impulsivo, con tutte le sue limitazioni.

■■ ATTENZIONE — ERRORE COMUNE ALL'ESAME

Un errore frequente è presentare il sistema impulsivo come un "difetto" da correggere e quello riflessivo come l'ideale a cui tendere. Non è così. Il sistema impulsivo è potentissimo e indispensabile: è lui che permette al programmatore esperto di "vedere" un pattern in pochi secondi, e senza di lui il lavoro creativo sarebbe impossibile. Il problema non è il sistema impulsivo in sé, ma il suo uso acritico in contesti che richiedono invece un'analisi esplicita. Il Motivated Tactician sa scegliere quale sistema attivare in base al contesto.

CAPITOLO 8

Mente Estesa, Cognizione Distribuita e Pensiero Sistemico

La mente non si ferma alla superficie del cranio

Nel 1998, i filosofi Andy Clark e David Chalmers pubblicarono un articolo intitolato "The Extended Mind" che destò grande interesse al di là dei confini accademici della filosofia della mente. La tesi centrale era provocatoria: i processi cognitivi non si concludono necessariamente all'interno del cranio. Possono estendersi nell'ambiente fisico e negli strumenti che usiamo, se questi soddisfano certi criteri funzionali.

L'esempio che usano è diventato celebre. Immaginate due persone che devono recarsi a un museo di New York. Inga, che non ha problemi di memoria, ricorda perfettamente dove si trova il museo: l'indirizzo è nella sua memoria biologica. Otto, che soffre di Alzheimer, porta con sé un taccuino su cui ha annotato tutto ciò che deve ricordare, incluso l'indirizzo del museo. Quando hanno bisogno dell'informazione, entrambi la recuperano: Inga accede alla sua memoria, Otto apre il taccuino. Clark e Chalmers sostengono che **funzionalmente non c'è differenza**: il taccuino di Otto è, a tutti gli effetti, la sua memoria. La distinzione tra "memoria nella testa" e "memoria nel mondo" è una distinzione fisica, non cognitiva.

Per lo sviluppo software, questa prospettiva ha implicazioni profonde. L'IDE non è solo uno strumento: è un'**estensione cognitiva** del programmatore. La documentazione, le note su carta, i diagrammi di architettura, Stack Overflow, il sistema di version control — tutto questo fa parte del processo cognitivo dello sviluppatore in modo funzionalmente equivalente alla sua memoria biologica. Un programmatore privato dei suoi strumenti non è semplicemente "meno produttivo": è cognitivamente menomato in senso reale. I moderni LLM e Copilot sono la più recente espansione di questa mente estesa: permettono di accedere a pattern e soluzioni che sarebbero inaccessibili alla sola memoria biologica del programmatore.

Vygotsky, la ZPD e il Pair Programming

Lo psicologo russo Lev Vygotsky sviluppò a inizio Novecento una teoria dell'apprendimento radicalmente diversa da quella prevalente all'epoca. Per Vygotsky, l'apprendimento non è un processo individuale: è intrinsecamente **sociale**. Apprendiamo attraverso l'interazione con altri, e in particolare attraverso la collaborazione con persone più competenti di noi.

Il concetto chiave è la **Zona di Sviluppo Prossimale (ZPD)**: lo spazio cognitivo tra ciò che un individuo può fare autonomamente e ciò che potrebbe fare con il supporto di un esperto o di un pari più avanzato. Vygotsky sosteneva che l'apprendimento avviene prevalentemente in questa zona: né nelle cose già padroneggiate (non c'è nulla da imparare), né in quelle troppo lontane dalle competenze attuali (non c'è supporto sufficiente per raggiungerle), ma precisamente in quella zona intermedia di sfida raggiungibile con aiuto.

Il concetto di **scaffolding** — impalcatura — descrive il tipo di supporto che l'esperto fornisce al novizio per permettergli di operare nella ZPD. Come l'impalcatura di un edificio in costruzione, questo supporto è

temporaneo: viene rimosso gradualmente man mano che il novizio acquisisce autonomia. Il Pair Programming di XP è fondato esattamente su questo principio: il driver e il navigator non si dividono semplicemente il lavoro, si supportano reciprocamente abbassando la ZPD di entrambi e accelerando l'apprendimento reciproco.

Il conformismo e i suoi rischi nei team di sviluppo

Il pensiero di Vygotsky suggerisce anche un rischio che il docente Succi considera di grande importanza etica: il conformismo cognitivo. Come dimostrano gli esperimenti classici di Solomon Asch e Stanley Milgram, la pressione sociale — del gruppo o dell'autorità — può spingere individui intelligenti e capaci a **accettare soluzioni che sanno essere errate**, o a compiere azioni che ritengono sbagliate, pur di conformarsi alle aspettative del gruppo o dell'autorità.

Nel contesto dello sviluppo software, il conformismo può manifestarsi in molti modi: un junior che non segnala un bug nell'architettura proposta dal senior per timore di sembrare arrogante; un team che non solleva obiezioni a una decisione del product owner perché la pressione temporale è alta; un'intera organizzazione che continua a usare una tecnologia obsoleta perché nessuno vuole essere il primo a dire che il re è nudo. Il pair programming e le retrospettive regolari, quando praticate in un clima psicologicamente sicuro, sono meccanismi di contrasto al conformismo cognitivo.

■ Intelligenza Artificiale e Neuroscienze

CAPITOLO 9

L'Intelligenza Artificiale applicata allo Sviluppo Software

Due modi di intendere l'AI nel corso

Il Prof. Succi adotta una prospettiva sull'Intelligenza Artificiale che è specifica di questo corso e che vale la pena comprendere bene, perché differisce dall'uso comune del termine. Nel corso, l'AI viene vista attraverso due assi concettuali: il **Modeling** (modellazione) e l'**Emulating** (emulazione) della mente umana.

Il **Modeling** riguarda l'uso dell'AI — e più in generale della tecnologia — come strumento per *capire come funziona la mente del programmatore*. In questa prospettiva, l'AI non sostituisce il programmatore: lo studia. Esempi concreti includono l'analisi dei segnali EEG per misurare il carico cognitivo durante la programmazione, l'analisi dei pattern di commit per predire la qualità del codice, o lo studio delle eye-tracking data per capire come i programmatori leggono il codice. L'obiettivo è costruire una scienza cognitiva del software: capire come funziona la mente umana quando produce software, per poi intervenire in modo informato sui processi e sugli strumenti.

L'**Emulating** è invece ciò a cui si pensa normalmente quando si parla di AI nello sviluppo software: algoritmi che replicano le azioni umane. Machine learning per la predizione dei bug, NLP per la generazione automatica di documentazione, LLM per il code completion, sistemi di analisi statica basati su reti neurali per il rilevamento di vulnerabilità. In questo caso, l'obiettivo è automatizzare o aumentare le capacità del programmatore, non studiarlo.

La distinzione è importante perché chiarisce *perché* il corso tratta argomenti apparentemente lontani dall'informatica come l'EEG e la psicologia cognitiva: sono strumenti del Modeling. L'AI che emula è inutile se non capiamo cosa sta emulando — e per capirlo, dobbiamo usare l'AI come strumento di modellazione della mente.

■ NOTA DI APPROFONDIMENTO

Un collegamento elegante che il docente apprezza: il Modeling AI corrisponde al "naive scientist" — cerca di capire come funziona davvero la mente. L'Emulating AI è invece il "cognitive miser" computazionale — usa pattern statistici per produrre risultati accettabili con il minimo sforzo computazionale. Gli LLM, in particolare, sono la più potente implementazione dell'abduzione batesoniana: trovano il pattern che meglio spiega il contesto dato.

CAPITOLO 10

I Segnali Cerebrali e l'EEG applicati alla Programmazione

Perché misurare il cervello del programmatore?

Se accettiamo la prospettiva del Modeling — cioè che l'AI può essere usata per capire come funziona la mente del programmatore — allora la domanda naturale è: quali strumenti possiamo usare per osservare la mente al lavoro? La risposta del gruppo di ricerca del Prof. Succi è l'**Elettroencefalografia (EEG)**.

L'EEG misura l'attività elettrica del cervello attraverso elettrodi posizionati sul cuoio capelluto. A differenza di altri strumenti di neuroimaging — come la fMRI (risonanza magnetica funzionale) o la PET (tomografia a emissione di positroni) — l'EEG ha due caratteristiche che lo rendono ideale per lo studio dei programmatori: è **portatile e relativamente economico**, e ha un'**altissima risoluzione temporale**. La fMRI produce immagini bellissime con alta risoluzione spaziale, ma misura variazioni del flusso sanguigno che avvengono su scale di secondi — troppo lento per catturare i dinamici processi cognitivi del debugging. L'EEG cattura variazioni nell'ordine dei millisecondi, perfetto per seguire il pensiero in tempo reale.

Gli elettrodi vengono posizionati secondo il sistema internazionale "10-20", che divide il cuoio capelluto in zone corrispondenti ai lobi cerebrali: prefrontale (Fp), frontale (F), temporale (T), parietale (P), occipitale (O) e centrale (C). Ogni elettrodo legge l'attività elettrica media della zona di corteccia sottostante, producendo un segnale continuo nel tempo.

Le onde cerebrali e il loro significato

Il segnale EEG viene analizzato scomponendolo nelle sue componenti di frequenza. Diverse bande di frequenza corrispondono a diversi stati cognitivi, e questa corrispondenza è alla base di tutta la ricerca EEG applicata alla programmazione.

Le onde **Delta** (1-5 Hz) sono associate al sonno profondo e all'inconscio. Le onde **Theta** (4-8 Hz) sono particolarmente interessanti: aumentano durante compiti cognitivi intensi, specialmente quelli che coinvolgono la memoria di lavoro e la pianificazione. Un programmatore che sta cercando di tenere mentalmente traccia di molte variabili durante il debugging tende a mostrare un aumento delle onde Theta. Le onde **Alpha** (8-12 Hz) rappresentano uno stato di rilassamento vigile: sono alte quando siamo svegli e tranquilli, e si *abbassano* quando iniziamo a impegnarci attivamente in un compito mentale. Questo abbassamento — chiamato **desincronizzazione Alpha** — è uno degli indicatori più affidabili di attivazione cognitiva. Le onde **Mu** (7.5-12.5 Hz) sono particolarmente interessanti per il Pair Programming: sono associate all'attività motoria e ai neuroni specchio, e si attivano sia quando eseguiamo un'azione sia quando la osserviamo in qualcun altro. Le onde **Beta** (12-25 Hz) sono associate alla concentrazione attiva, al pensiero analitico e all'attenzione focalizzata.

ERD, ERS e la misurazione del carico cognitivo

I due indicatori EEG più importanti per la ricerca sulla programmazione sono l'**ERD** (Event-Related Desynchronization) e l'**ERS** (Event-Related Synchronization). L'ERD misura una *diminuzione* dell'attività oscillatoria in risposta a un evento o a un compito: riflette che il cervello sta attivamente elaborando informazioni, rompendo la sincronia a riposo per dedicare risorse al compito. L'ERS misura invece un *aumento* dell'attività oscillatoria: il cervello è relativamente a riposo, i neuroni si sincronizzano.

La formula per calcolare l'ERD/ERS è la seguente: si misura l'ampiezza del segnale nella banda di interesse durante un periodo di riferimento (il programmatore è a riposo), poi si misura la stessa ampiezza durante il compito di programmazione, e si calcola la variazione percentuale.

$$\text{ERD} = (\text{Ampiezza_Riposo} - \text{Ampiezza_Programmazione}) / \text{Ampiezza_Programmazione} \times 100\%$$

Se $ERD > 0 \rightarrow$ desincronizzazione $\rightarrow$ il cervello è attivamente impegnato (problem solving)

Se $ERD < 0$ (ERS) $\rightarrow$ sincronizzazione $\rightarrow$ il cervello è relativamente a riposo

Un aspetto tecnico importante è il calcolo dell'**IAF (Individual Alpha Frequency)**: la frequenza Alpha non è la stessa per tutti. Diminuisce con l'età, e varia da individuo a individuo. Per questo ogni partecipante agli studi EEG viene analizzato con la propria frequenza Alpha individuale, calcolata con la formula: $IAF = 11.95 - (0.053 \times Et\grave{a})$. Questo garantisce che il segnale di interesse venga analizzato nella banda giusta per ogni persona.

La **pipeline di analisi MNE-Python** segue questi passi: lettura del file dati grezzi (`read_raw_edf`); applicazione di un filtro notch a 50 Hz per rimuovere il rumore introdotto dall'alimentazione elettrica europea; calcolo dell'IAF individuale; filtraggio bandpass per isolare la banda di interesse; elevamento al quadrato dei campioni per ottenere la potenza del segnale; mediazione su tutte le trial per ridurre il rumore; infine, calcolo dell'ERD/ERS come variazione percentuale rispetto al periodo di riferimento.

■ COME RISPONDERE ALL'ESAME

Le domande sull'EEG si aspettano che tu mostri di capire *perché* si usa l'EEG (non la fMRI), cosa misura l'ERD/ERS (sforzo cognitivo), e *come* si collega alla ricerca cognitiva sulla programmazione. Il collegamento più apprezzato è: l'EEG permette di misurare oggettivamente il carico cognitivo del programmatore, fornendo dati empirici per validare i modelli teorici come il Modello Impulsivo/Riflessivo. Si chiude il cerchio: la teoria cognitiva ispira la ricerca EEG, e i dati EEG validano (o falsificano) la teoria.

■ ■ Blockchain, Agile e Change Management

CAPITOLO 11

Blockchain come Sistema Ingegneristico

Una prospettiva diversa sulla blockchain

Nel contesto di questo corso, la blockchain non viene presentata principalmente come una tecnologia finanziaria o come la base delle criptovalute, ma come un **sistema ingegneristico con proprietà di sistema dinamico** studiate nei capitoli precedenti. In particolare, la blockchain è un tentativo di costruire artificialmente un regime Pseudo-Deterministico (PDS) in un ambiente naturalmente caotico e distribuito, eliminando quello che gli ingegneri chiamano "single point of failure".

Nei sistemi centralizzati tradizionali, esiste sempre un nodo centrale — un server, un database, un'autorità — la cui compromissione o il cui guasto mette a rischio l'intero sistema. Questo è il single point of failure. La blockchain risolve questo problema distribuendo il registro delle transazioni su migliaia di nodi indipendenti: non esiste un centro da attaccare o un'autorità da corrompere. La **fiducia** non è delegata a un'istituzione, ma è incorporata nel protocollo matematico stesso.

La struttura di un blocco nella blockchain riflette questa logica. Ogni blocco contiene i dati della transazione, un **timestamp**, l'**hash del blocco precedente** (che crea la catena e rende impossibile modificare un blocco senza invalidare tutti quelli successivi), un **nonce** (un numero trovato attraverso il processo di mining che rende la creazione di un blocco computazionalmente costosa e quindi difficile da falsificare) e il proprio **hash**. Questa struttura garantisce l'**immutabilità** del registro: aggiungere dati è facile, modificarli è praticamente impossibile.

Usando il linguaggio del corso, la blockchain è un sistema a **retroazione negativa**: ogni tentativo di destabilizzare il registro (attacco, modifica fraudolenta) viene contrastato dal consenso degli altri nodi, che rifiutano la versione modificata. Il protocollo di consenso — sia esso Proof of Work o Proof of Stake — è il meccanismo di retroazione che mantiene il sistema nel regime PDS desiderato. Ma, come tutti i sistemi PDS, la blockchain nasconde fragilità: un attacco del 51%, un bug critico negli smart contract, o una fork nella community possono spingere il sistema verso un regime DCS o SR.

Blockchain e AI: un collegamento naturale

Il collegamento più interessante tra AI e blockchain, nell'ottica del corso, riguarda gli **smart contract**. Gli smart contract sono programmi che vivono sulla blockchain e si eseguono automaticamente al verificarsi di certe condizioni. Il problema è che, una volta deployati, non possono essere modificati — la loro immutabilità è la stessa della blockchain su cui vivono. Questo crea una tensione profonda con i principi Agile: come si gestisce un problema Wicked (il cui requisiti cambiano durante lo sviluppo) con uno strumento che è per definizione immutabile una volta rilasciato?

L'AI come Modeling può essere usata per analizzare il codice degli smart contract *prima* del deploy, cercando vulnerabilità e pattern di codice problematici. Questo sposta il problema dal regime SR (la vulnerabilità viene scoperta dopo il deploy, quando è irreversibile) al regime StS (la vulnerabilità viene rilevata con una certa probabilità prima del deploy, quando è ancora correggibile). È un esempio concreto di come AI, blockchain e principi di ingegneria del software si integrano in modo non banale.

CAPITOLO 12

Lean, Agile ed Extreme Programming

Da dove vengono le metodologie agili?

Le metodologie agili non sono nate dal nulla nel 2001 con la pubblicazione del Manifesto Agile. Affondano le radici in decenni di ricerca sui limiti del modello Waterfall, nell'esperienza pratica di team di sviluppo che avevano imparato sulla propria pelle che la pianificazione rigida non funzionava per i problemi Wicked, e nei principi del Lean Manufacturing giapponese. Comprendere questa genesi è essenziale per rispondere in modo profondo alle domande del docente.

Il **Lean Manufacturing** nacque in Toyota negli anni '50 come risposta alla necessità di competere con la produzione di massa americana pur avendo risorse molto più limitate. Il principio fondamentale del Lean è l'eliminazione degli sprechi — in giapponese, **Muda**. Qualsiasi attività che non produce valore diretto per il cliente è uno spreco e deve essere eliminata o ridotta. Mary e Tom Poppendieck adattarono questi principi al software nel 2003, identificando sette categorie di sprechi tipici dello sviluppo.

Il primo spreco è il **Lavoro Parziale**: codice scritto ma non integrato, non testato, non deployato. Un modulo completato al 90% non ha valore — non è un asset, è un costo sospeso che continua ad accumularsi. Il secondo è i **Processi Extra**: tutta la burocrazia, la documentazione che nessuno leggerà mai, le riunioni senza agenda. Il terzo è la produzione eccessiva di **Funzionalità**: ricerche empiriche mostrano che circa il 64% delle funzionalità sviluppate non viene quasi mai usato dagli utenti. Sviluppare "nel caso serva" è uno spreco enorme. Il quarto è il **Task Switching**: passare continuamente tra progetti diversi distrugge la profondità del contesto cognitivo, e ogni cambio di contesto richiede un "warm-up" cognitivo costoso — misurabile anche via EEG. Le **Attese** (quinto spreco) sono i colli di bottiglia: PR che aspettano review, ticket in attesa di approvazione. Il **Movimento** (sesto spreco) riguarda i passaggi informativi inefficienti. I **Difetti** (settimo spreco) scoperti tardi — in produzione invece che durante lo sviluppo — costano esponenzialmente di più di quelli scoperti presto.

Il Manifesto Agile e i suoi principi

Nel febbraio del 2001, diciassette sviluppatori si riunirono a Snowbird, Utah, e firmarono un documento destinato a cambiare il modo di pensare allo sviluppo software. Il Manifesto Agile non è una metodologia: è una dichiarazione di valori. "Individui e interazioni" sopra processi e strumenti. "Software funzionante" sopra documentazione esaustiva. "Collaborazione con il cliente" sopra negoziazione dei contratti. "Rispondere al cambiamento" sopra seguire un piano. Notare che il manifesto non nega il valore delle cose a destra: dice che quelle a sinistra hanno più valore. Non butta via la documentazione, ma dice che il software funzionante viene prima.

Extreme Programming: le pratiche e il loro fondamento cognitivo

L'**Extreme Programming (XP)**, sviluppato da Kent Beck, è la metodologia Agile più radicale e quella che il corso approfondisce maggiormente, perché ogni sua pratica può essere spiegata in termini cognitivi usando i modelli trattati nelle parti precedenti.

Il **Pair Programming** non è solo un modo di condividere la conoscenza. È un'implementazione del coordinamento per risultato comune di Malone, costruita sulla ZPD di Vygotsky (driver e navigator si supportano reciprocamente abbassando la ZPD di entrambi), e un meccanismo per attivare sistematicamente il sistema riflessivo: mentre il driver scrive, il navigator mantiene attiva la visione d'insieme con il sistema riflessivo, impedendo che la concentrazione sul dettaglio (sistema impulsivo) faccia perdere di vista l'architettura generale.

Il **Test-Driven Development (TDD)** è un ciclo di feedback brevissimo che impone di scrivere prima il test e poi il codice. Questo non è solo una tecnica di testing: è un modo di forzare il programmatore a pensare all'interfaccia e al comportamento atteso prima di immergersi nell'implementazione. Attiva il sistema riflessivo nella fase di progettazione, invece di lasciare che il sistema impulsivo guidi direttamente verso la prima implementazione che viene in mente.

Il **refactoring continuo** e la **Continuous Integration** sono risposte dirette al settimo spreco del Lean (i difetti scoperti tardi) e alla terza difficoltà di Brooks (la modificabilità). Integrando frequentemente e tenendo il codice sempre "pulito", si mantiene il sistema nel regime StS — controllabile statisticamente — invece di lasciarlo accumulare debito tecnico fino a che il regime PDS non collassa in un episodio DCS.

CAPITOLO 13

Il Change Management nello Sviluppo Software

Perché il cambiamento è al centro di tutto

Uno dei concetti più pervasivi dell'intero corso è quello del cambiamento. Il software deve cambiare perché i requisiti cambiano. I team devono cambiare perché le persone crescono e i contesti evolvono. Le organizzazioni devono cambiare perché il mercato cambia. Eppure, come abbiamo visto studiando l'homeostasis di Weinberg, tutti i sistemi tendono a resistere al cambiamento. Gestire questa tensione è una delle competenze fondamentali dell'ingegnere del software.

Il docente cita il famoso paradosso attribuito a Giuseppe Tomasi di Lampedusa nel romanzo "Il Gattopardo": *"Se vogliamo che tutto rimanga come è, bisogna che tutto cambi."* Nel software, questa paradossale saggezza si traduce così: per mantenere un sistema funzionante, rilevante e manutenibile nel tempo, è necessario cambiarlo continuamente. Un sistema che non si evolve non rimane fermo: degrada progressivamente. Il refactoring, gli aggiornamenti delle dipendenze, la riscrittura di componenti obsoleti non sono "sprechi" — sono la manutenzione preventiva che impedisce al sistema di collassare.

Le strategie di gestione del cambiamento si articolano su tre livelli. Si può cercare di **prevenire** il cambiamento attraverso requisiti dettagliati e stabili — strategia che funziona solo per i problemi Tame, ma che viene spesso applicata anche ai Wicked con risultati disastrosi. Si può **anticipare** il cambiamento attraverso la domain analysis (analisi del dominio applicativo) e la definizione di variation points (punti di variabilità nell'architettura): si progetta il sistema sapendo che certe cose cambieranno, e si creano "giunture" nell'architettura che permettono di evolvere quelle parti senza toccare il resto. Infine, si può **gestire** il cambiamento attraverso la manutenzione sistematica: correttiva (fix di bug), perfettiva (miglioramento della qualità interna, incluso il refactoring), adattiva (adattamento a nuovi ambienti, sistemi operativi, normative) e

preventiva (migliorare la manutenibilità futura prima che sorga un problema).

■■ ATTENZIONE — ERRORE COMUNE ALL'ESAME

Il docente usa esempi storici e letterari per mostrare che "change" è spesso invocato come buzzword per giustificare azioni non etiche o per mascherare l'immobilismo. Cita Mussolini ("il fascismo è il cambiamento") come esempio di come il linguaggio del cambiamento possa essere svuotato di significato e strumentalizzato. La risposta ideale all'esame mostra consapevolezza critica: il cambiamento non è un valore assoluto, ma deve essere valutato in termini di chi beneficia, di cosa cambia davvero e di quali conseguenze produce.

■ Simulazione d'Esame e Piano di Studio

CAPITOLO 14

Piano di Studio per 3 Giorni

L'esame del Prof. Succi è un orale onnicomprensivo. Non si testa la memorizzazione di definizioni, ma la capacità di ragionare sui concetti e di collegarli tra loro. Il piano qui sotto è costruito su questo presupposto: il primo giorno costruisce la comprensione, il secondo consolida i collegamenti, il terzo simula la performance orale.

■ GIORNO 1 (Domenica) — COMPRENSIONE

Mattina: Leggi con attenzione i Capitoli 1, 2 e 3 di questo manuale (software, Brooks, regimi di Mensch). Per ogni concetto, fermati e prova a spiegarlo a voce alta come se fossi all'esame. Disegna su carta la "mappa" dei regimi PDS/DCS/StS/SR con esempi tuoi.

Pomeriggio: Capitoli 4, 5, 6 e 7 (Tame/Wicked, Harris, Bateson, modelli cognitivi). Il modello Impulsivo/Riflessivo merita attenzione speciale: rileggilo più volte e cerca almeno 3 esempi dalla tua esperienza personale di programmazione.

Sera: Capitoli 8 e 9 (Mente estesa, EEG). Memorizza la formula ERD/ERS e la pipeline MNE-Python. A fine giornata, scrivi 2-3 pagine a mano (sì, a mano — come raccomanda il docente) riassumendo i concetti che ancora non ti sono chiari.

■ GIORNO 2 (Lunedì) — COSTRUZIONE DEI COLLEGAMENTI

Mattina: Capitoli 10, 11, 12 e 13 (Blockchain, Lean, XP, Change). Per ogni concetto, chiediti: "Come si collega ai modelli cognitivi?" e "In quale regime di Mensch si inserisce?". Scrivi le risposte.

Pomeriggio: Esercizio fondamentale — disegna la mappa concettuale completa del corso su un foglio grande. Parti da "software come atto creativo" e traccia le frecce di collegamento. Cerca almeno 5 percorsi che attraversano tutti e quattro i macro-temi del corso.

Sera: Esercitati con domande trasversali: "Spiega il Pair Programming usando almeno tre teorie diverse." "Perché l'Agile funziona sui problemi Wicked?" "Come risponde la blockchain al problema del single point of failure in termini di regimi di Mensch?"

■ GIORNO 3 (Martedì) — SIMULAZIONE ORALE

Mattina: Leggi le 10 domande della sezione successiva una per una. Per ognuna, rispondi ad alta voce — o scrivi a mano — come se fossi all'esame. Cronometrati: una risposta orale ideale dura 3-5 minuti. Né troppo corta (mancanza di profondità), né troppo lunga (perdita di filo logico).

Pomeriggio: Revisiona i punti deboli emersi dalla mattina. Rileggili dal manuale, non da appunti sintetici. La comprensione profonda richiede testo narrativo, non bullet points.

Sera: Smetti di studiare entro le 20:00. Mangia bene, dormi almeno 7-8 ore. Il cervello consolida la memoria durante il sonno: l'ultima notte è più importante dell'ultima ora di studio.

CAPITOLO 15

10 Domande d'Esame con Risposte Elaborate

Le seguenti domande e risposte sono costruite per rispecchiare lo stile dell'esame orale del Prof. Succi: domande che chiedono di spiegare, collegare e ragionare — non di recitare definizioni. Le risposte modello mostrano il livello di profondità e di collegamento interdisciplinare atteso.

DOMANDA 1 — Perché il software è protetto dal diritto d'autore e non dal brevetto? Quali implicazioni ha questa scelta?

Il software è il prodotto di un atto creativo intellettuale, assimilabile a un'opera letteraria o musicale più che a un'invenzione tecnica. Il diritto d'autore nasce automaticamente con la creazione dell'opera, senza bisogno di registrazione, e dura 50 anni dopo la morte dell'autore — molto più dei 20 anni di un brevetto. Questa protezione si articola in diritti morali, che sono non alienabili: un programmatore non può cedere il diritto di essere riconosciuto come autore della propria opera, anche se lavora per un'azienda. I diritti economici, invece, sono alienabili: lo sfruttamento commerciale può essere ceduto a terzi, come avviene tipicamente nei contratti di lavoro dipendente. La conseguenza pratica più rilevante è che il software non viene mai "venduto" all'utente finale: viene concesso in licenza. Acquistare un software significa acquistare il diritto di usarlo secondo certi termini, non la proprietà del codice. Questa distinzione ha implicazioni profonde per il modello economico del software: combinata con il costo marginale di replica praticamente nullo (funzione di costo a "L"), crea le condizioni per strategie aggressive di segmentazione del mercato e bundling che non hanno equivalenti nell'industria manifatturiera tradizionale.

DOMANDA 2 — Descriva i quattro regimi di Mensch e spieghi perché il software si trova quasi sempre nei regimi DCS o StS.

Mensch classifica i sistemi dinamici in quattro regimi a seconda della loro prevedibilità strutturale. Il regime Pseudo-Deterministico (PDS) appare stabile perché è mantenuto tale da meccanismi di controllo ingegneristici o istituzionali, ma nasconde fragilità latenti — è l'"illusione del tacchino": il sistema sembra deterministico finché una perturbazione fuori dall'ordinario ne rivela la fragilità. Il regime Caotico Deterministico (DCS) è governato da equazioni precise, ma presenta sensibilità estrema alle condizioni iniziali: piccole variazioni producono risultati radicalmente diversi nel lungo periodo (effetto farfalla). Il regime Stocastico (StS) ha incertezza strutturale, ma mantiene una distribuzione probabilistica stabile che permette stime ragionevoli. Il regime di Randomicità Stocastica (SR) è caos puro: la previsione è impossibile. Il software si trova quasi sempre nei regimi DCS o StS per ragioni intrinseche alla sua natura. È un problema Wicked: i requisiti cambiano durante lo sviluppo, e ogni implementazione rivela nuove esigenze. È costruito da esseri umani che operano secondo il modello cognitivo del Motivated Tactician, introducendo variabilità non pianificata. Gira su sistemi hardware e di rete con comportamenti difficilmente prevedibili in tutte le loro interazioni. Il Waterfall assume implicitamente il regime PDS e fallisce perché la sua assunzione è quasi sempre sbagliata. L'Agile è la risposta corretta per DCS/StS: invece di pianificare rigidamente, gestisce l'incertezza attraverso cicli brevi di feedback.

DOMANDA 3 — Descriva il modello Impulsivo/Riflessivo di Strack e Deutsch e le sue implicazioni per le code review.

Strack e Deutsch (2004) descrivono il comportamento umano come risultato dell'interazione di due sistemi cognitivi paralleli. Il Sistema Impulsivo è implicito, automatico e velocissimo: opera in millisecondi sulla base di una memoria associativa costruita dall'esperienza, senza richiedere alcuno sforzo consapevole. Riconosce pattern, produce intuizioni, genera risposte immediate. Il Sistema Riflessivo è esplicito, deliberato e lento: richiede attenzione focalizzata e sforzo cognitivo, ma è capace di ragionamenti formali, analisi sistematiche e valutazioni di lungo periodo. I due sistemi operano in parallelo e a volte producono risultati contraddittori: è il conflitto tra il dolce che si vuole mangiare (sistema impulsivo) e la dieta che si vuole rispettare (sistema riflessivo). L'implicazione per le code review è profonda e confermata da ricerca empirica. Quando un revisore legge il codice sotto pressione temporale — come accade quasi sempre nel ciclo di sviluppo reale — il sistema impulsivo tende a dominare: si "sente" se il codice va bene o no, si cerca il pattern riconoscibile, si formula un giudizio rapido. Il sistema riflessivo non si attiva automaticamente, e senza di lui le review trovano solo i bug che corrispondono a pattern già noti. Per migliorare la qualità delle review è necessario progettare il processo in modo da forzare l'attivazione del sistema riflessivo: checklist esplicite che costringono ad analizzare sistematicamente ogni aspetto, tempo adeguato senza pressione, separazione tra la fase di lettura e quella di valutazione, revisioni a coppie che obbligano a verbalizzare il ragionamento.

DOMANDA 4 — Cosa significa che la mente è "estesa" secondo Clark e Chalmers? Come si collega al Pair Programming e agli LLM?

Clark e Chalmers (1998) sostengono che i processi cognitivi non si concludono necessariamente all'interno del cranio: possono estendersi nell'ambiente e negli strumenti, se questi soddisfano certi criteri funzionali. Il loro esempio è Otto, malato di Alzheimer, che usa un taccuino per compensare la perdita di memoria. Clark e Chalmers argomentano che funzionalmente il taccuino di Otto è equivalente alla memoria biologica di Inga: entrambi sono sistemi di recupero dell'informazione che supportano il comportamento cognitivo. La differenza è solo nel supporto fisico, non nella funzione. Per lo sviluppo software, le implicazioni sono immediate: l'IDE, la documentazione, i diagrammi di architettura, Stack Overflow, il version control non sono semplici strumenti — sono estensioni funzionali della mente del programmatore. Privare un programmatore del suo ambiente di lavoro è una menomazione cognitiva reale, non solo una riduzione di produttività. Il Pair Programming sfrutta la mente estesa in modo esplicito: il partner è un'estensione cognitiva che abbassa la ZPD (Vygotsky) di entrambi e che mantiene attivo il sistema riflessivo del driver mentre questi è concentrato sull'implementazione con il sistema impulsivo. Gli LLM — Copilot, Claude, ChatGPT — sono la più recente e potente espansione della mente estesa: permettono di accedere a pattern e soluzioni che sarebbero inaccessibili alla memoria biologica del singolo programmatore, realizzando computazionalmente il processo di abduzione batesoniana.

DOMANDA 5 — Descriva il caso Ariane 5 e le lezioni che se ne traggono per l'ingegneria del software.

Il disastro di Ariane 5 del 4 giugno 1996 è uno degli esempi più citati nella letteratura di ingegneria del software per la sua chiarezza didattica. Il vettore spaziale europeo, al suo volo inaugurale, si autodistrusse 37 secondi dopo il lancio a causa di un difetto software. Un modulo del sistema di riferimento inerziale era stato preso direttamente dal vettore precedente, Ariane 4, dove funzionava correttamente da anni. Il problema era che Ariane 5 aveva una traiettoria di lancio diversa, con velocità orizzontali significativamente più alte. Queste velocità più alte producevano valori numerici che il codice di Ariane 4 non era stato progettato per gestire: un numero in virgola mobile a 64 bit veniva inserito in una variabile a 16 bit, producendo un overflow. Il modulo mandò un messaggio diagnostico di errore all'unità di controllo principale, che interpretò questo messaggio come dati di navigazione validi, generando comandi di correzione assurdi. Il razzo si spezzò e attivò il sistema di autodistruzione. Il danno economico fu stimato intorno agli 8 miliardi di dollari. Le lezioni per l'ingegneria del software sono multiple e profonde. Prima lezione: il riutilizzo del software è desiderabile ma non è mai gratuito. Un modulo che funziona in un contesto non funziona necessariamente in un altro — la seconda difficoltà di Brooks (Conformità) si applica anche al riutilizzo interno. Seconda lezione: le assunzioni implicite nel codice — i valori che si presuppone non possano mai essere troppo grandi, le condizioni che si presuppone siano sempre soddisfatte — sono una fonte primaria di difetti catastrofici. Terza lezione: la quarta difficoltà di Brooks (Invisibilità) rende difficile "vedere" queste assunzioni implicite, specialmente quando il codice è riusato senza una comprensione profonda.

DOMANDA 6 — Cos'è la Zona di Sviluppo Prossimale di Vygotsky? Come si manifesta nel Pair Programming?

Lev Vygotsky sviluppò una teoria dell'apprendimento radicalmente sociale: per lui, il pensiero e l'apprendimento non sono processi individuali, ma si formano nell'interazione con gli altri. Il concetto chiave è la Zona di Sviluppo Prossimale (ZPD): lo spazio cognitivo tra ciò che un individuo riesce a fare autonomamente e ciò che potrebbe fare con il supporto di un esperto o di un pari più avanzato. Vygotsky sostiene che l'apprendimento avviene principalmente in questa zona: le attività già padroneggiate non producono crescita, quelle troppo distanti dalle competenze attuali non sono raggiungibili neppure con aiuto, ma quelle nella ZPD — sfidanti ma raggiungibili con supporto — sono il motore dello sviluppo cognitivo. Il concetto di scaffolding descrive il tipo di supporto che permette di operare nella ZPD: come le impalcature di un edificio, è temporaneo e viene rimosso man mano che il novizio acquisisce autonomia. Nel Pair Programming questo meccanismo è sistematizzato: il driver — che scrive il codice — e il navigator — che mantiene la visione d'insieme — si supportano reciprocamente, abbassando la ZPD di entrambi. Il junior impara dal senior, ma anche il senior beneficia della prospettiva fresca del junior, che a volte nota cose che l'esperienza ha reso invisibili. Il Pair Programming è quindi al tempo stesso: un meccanismo di coordinamento per risultato comune (Malone), un'implementazione della ZPD (Vygotsky), e un sistema per mantenere attivo il sistema riflessivo durante la scrittura del codice (Strack e Deutsch).

DOMANDA 7 — Descriva l'ERD nell'EEG. Cosa ci dice sull'attività cognitiva di un programmatore durante il debugging?

L'ERD, Event-Related Desynchronization, è uno dei principali indicatori nell'analisi EEG dello stato cognitivo. Si riferisce a una diminuzione dell'attività oscillatoria cerebrale — tipicamente nelle bande Alpha o Beta — in risposta a un compito cognitivo. Il termine "desincronizzazione" descrive il fatto che i neuroni, che a riposo tendono a oscillare in modo sincrono (producendo un segnale EEG ampio e regolare), durante l'elaborazione attiva si "desincronizzano" — ognuno segue il proprio ritmo — producendo un segnale di ampiezza ridotta ma informativamente più ricco. La formula per misurarlo è: $ERD = (Ampiezza\ nel\ periodo\ di\ riferimento - Ampiezza\ durante\ il\ compito) / Ampiezza\ durante\ il\ compito \times 100\%$. Un valore positivo indica desincronizzazione — sforzo cognitivo attivo. L'ERS (Event-Related Synchronization) con valore negativo indica sincronizzazione — stato di riposo. Applicato al debugging, questo strumento rivela dinamiche cognitive affascinanti. Quando un programmatore si trova davanti a un bug difficile, si osserva tipicamente un aumento della potenza Theta (che riflette l'alto carico sulla memoria di lavoro necessario per tenere traccia di tutte le variabili rilevanti) e un ERD Alpha significativo (che indica l'intensa elaborazione attiva). Man mano che la causa del bug diventa chiara, il pattern cambia: si osserva spesso un'onda Theta che diminuisce (il problema mentale si semplifica) e talvolta un picco di attività nelle aree frontali associato all'insight. Questi dati permettono di costruire modelli quantitativi del carico cognitivo del programmatore — uno degli obiettivi del filone AI-as-Modeling del corso.

DOMANDA 8 — Descriva i 7 sprechi del Lean Software Development e colleghi uno di essi alla teoria cognitiva.

Il Lean Software Development, adattato da Mary e Tom Poppendieck dai principi del Toyota Production System, identifica sette categorie di attività che non producono valore per il cliente e che devono essere eliminate. Il Lavoro Parziale è il codice scritto ma non integrato, testato o deployato: non è un asset ma un costo sospeso. I Processi Extra includono tutta la burocrazia inutile: documentazione che nessuno leggerà, riunioni senza scopo, approvazioni ridondanti. Le Funzionalità Extra rappresentano il problema dell'overproduction: ricerche empiriche mostrano che circa il 64% delle funzionalità sviluppate non viene quasi mai usato. Il Task Switching è il costo del cambiamento continuo di contesto cognitivo. Le Attese sono i colli di bottiglia in qualsiasi parte del flusso. Il Movimento riguarda i passaggi informativi inefficienti. I Difetti scoperti tardi costano esponenzialmente di più di quelli trovati presto. Il Task Switching (quarto spreco) è quello con il collegamento più diretto alla teoria cognitiva del corso. Secondo la teoria della Mente Estesa di Clark e Chalmers, il programmatore che lavora su un problema costruisce un contesto cognitivo distribuito: nella testa, nell'IDE aperto, nelle finestre del browser, nelle note scritte, nei file modificati. Questo contesto è lo scaffold che supporta il suo lavoro. Quando il programmatore cambia task, questo scaffold deve essere smontato e ricostruito da zero per il nuovo problema. Il "warm-up" cognitivo — il tempo necessario per rientrare pienamente nel contesto di un problema dopo una interruzione — è reale, misurabile via EEG (si osserva un picco di ERD all'inizio di ogni nuovo task), e può durare dai 15 ai 25 minuti per problemi complessi. Questo è il motivo per cui la pratica XP di lavorare su un solo task alla volta ha una fondazione neuroscientifica, non solo organizzativa.

DOMANDA 9 — Come si integra la blockchain con i principi di ingegneria del software trattati nel corso?

Nel contesto di questo corso, la blockchain viene analizzata non come tecnologia finanziaria ma come sistema ingegneristico con proprietà definibili in termini dei concetti già studiati. La blockchain è un sistema decentralizzato a retroazione negativa che tenta di costruire artificialmente un regime Pseudo-Deterministico (PDS) in un ambiente distribuito e potenzialmente ostile, eliminando il singolo punto di fallimento attraverso la distribuzione del registro su migliaia di nodi indipendenti. Il protocollo di consenso — sia Proof of Work, sia Proof of Stake — è il meccanismo di retroazione negativa che mantiene il sistema nel regime PDS desiderato: ogni tentativo di modificare il registro in modo fraudolento viene rigettato dalla maggioranza dei nodi, che mantengono la versione autentica. Ma, come tutti i sistemi PDS, la blockchain può nascondere fragilità: un attacco del 51%, un bug critico negli smart contract, o una divisione nella community possono spingere il sistema verso DCS o SR. L'integrazione con l'AI è particolarmente rilevante nel caso degli smart contract. Questi programmi, una volta deployati, non possono essere modificati — la loro immutabilità è una proprietà fondamentale. Questo crea una tensione profonda con i principi Agili: come si gestisce un problema Wicked con uno strumento immutabile? L'AI come Modeling può analizzare il codice degli smart contract prima del deploy, cercando pattern di vulnerabilità. Questo sposta il rischio dal regime SR (vulnerabilità scoperta dopo il deploy, quando è irreversibile) al regime StS (vulnerabilità rilevata con una certa probabilità prima del deploy, quando è ancora correggibile). La blockchain è quindi anche un esempio di problema Wicked che richiede approccio Clan/Group per la sua governance — non esiste un'autorità centrale che decide, ma una comunità che si auto-organizza.

DOMANDA 10 — Descrivi la Matrice di Harris. Perché è rilevante per scegliere un approccio di sviluppo software?

La Matrice di Harris classifica i meccanismi di controllo di un processo in tre categorie, sulla base di quanto il processo stesso è conoscibile e prevedibile. Il controllo di tipo Outcome misura il risultato finale: funziona bene quando si conosce la relazione causa-effetto tra le azioni del team e il risultato desiderato. Un product manager che guarda il numero di bug risolti per sprint sta usando un controllo Outcome. Il controllo di tipo Behavior misura i passaggi intermedi del processo: funziona bene quando il processo è ben noto e standardizzabile, essenzialmente in un contesto Taylorista. Misurare le ore di lavoro, i commit al giorno, la copertura del codice sono esempi di controllo Behavior. Il controllo di tipo Clan/Group rinuncia a qualsiasi misurazione diretta e si affida invece ai valori condivisi, alla fiducia reciproca e all'auto-organizzazione del team. La rilevanza per lo sviluppo software diventa evidente connettendo la matrice alla distinzione Tame/Wicked e ai regimi di Mensch. I problemi Tame in regime PDS si prestano bene al controllo Behavior: il processo è prevedibile, le azioni producono risultati prevedibili, e la misurazione del processo ha senso. I problemi Wicked in regime StS o DCS non possono essere gestiti con controllo Behavior: le azioni del team non hanno una relazione prevedibile con il risultato, perché il risultato stesso non è definito in anticipo. In questi casi, il controllo Clan/Group è l'unico efficace. Questo spiega teoricamente perché le pratiche Agili — daily stand-up, retrospettive, pair programming — producano risultati positivi: non misurano, ma creano le condizioni cognitive e relazionali (fiducia, allineamento, psychological safety) che permettono di navigare l'incertezza. Il fenomeno di Goodhart — "quando una misura diventa un obiettivo, cessa di essere una buona misura" — descrive cosa succede quando si applica il controllo Behavior a un processo Wicked: il team ottimizza la metrica invece del valore reale.